i_05 リファクタリング実行手順書

目的

ファイル移動を安全に実行するための手順書。間違えた場合の戻し方も定義。

前提条件

- mainブランチから `refactor/3layer-architecture` ブランチを作成
- 全テストのベースライン記録済み
- i_01〜i_04の設計書レビュー済み

実行順序（依存関係順）

リファクタリングは依存関係の「根」から「葉」に向かって実行する。
先に移動すべきは「他から参照されるもの」。

Step 1: Manager層 + Engine層の移動（最優先）

全パイプラインから参照されるため、最初に移動。

```
1.1 workers/bpo/manager/ → workers/manager/
1.2 workers/bpo/engine/ → workers/engine/
1.3 全ファイルの import を sed で一括置換
1.4 テスト実行（manager + engine テストのみ）
```

この時点で壊れるファイル: 全パイプライン（56本）+ ルーター + main.py
修正方法: import パスの置換のみ（ロジック変更なし）

sedコマンド例:

```
# manager
find shachotwo-app/ -name "*.py" -exec sed -i '' \
  's/from workers\.bpo\.manager/from workers.manager/g' {} +
find shachotwo-app/ -name "*.py" -exec sed -i '' \
  's/import workers\.bpo\.manager/import workers.manager/g' {} +

# engine
find shachotwo-app/ -name "*.py" -exec sed -i '' \
  's/from workers\.bpo\.engine/from workers.engine/g' {} +
find shachotwo-app/ -name "*.py" -exec sed -i '' \
  's/import workers\.bpo\.engine/import workers.engine/g' {} +
```

検証コマンド:

```
cd shachotwo-app
python -c "from workers.manager.task_router import route_and_execute"
python -c "from workers.engine.models import PipelineResult"
pytest tests/workers/manager/ tests/workers/bpo/manager/ -v --tb=short
```

コミット: `refactor: move manager/ and engine/ out of bpo/ (Step 1/8)`

Step 2: micro層の業界中立化

compliance.pyから業界ルールを分離。

```
2.1 workers/industry/construction/rules/compliance_rules.py 新規作成（建設業法ルール）
2.2 workers/industry/manufacturing/rules/compliance_rules.py 新規作成（ISO規則）
2.3 micro/compliance.py から _CONSTRUCTION_RULES, _MANUFACTURING_RULES を削除
2.4 micro/compliance.py に汎用 check_rules(data, rules) インターフェースを定義
2.5 建設パイプラインが rules/compliance_rules.py を読んで micro/compliance に渡すように変更
2.6 テスト実行（micro + 建設 + 製造テスト）
```

注意: この Step は Step 3（業界プラグイン移動）より先に実行する。
理由: 業界固有コードを分離してからでないと、移動先の構造が確定しない。

検証コマンド:

```
cd shachotwo-app
python -c "from workers.micro.compliance import check_rules"
pytest tests/workers/micro/ -v --tb=short
pytest tests/workers/bpo/construction/ -v --tb=short
pytest tests/workers/bpo/manufacturing/ -v --tb=short
```

コミット: `refactor: extract industry rules from micro/compliance.py (Step 2/8)`

Step 3: 業界プラグインの移動（Layer B）

他から参照されないため、安全に移動可能。

```
3.1 workers/bpo/construction/ → workers/industry/construction/
3.2 workers/bpo/manufacturing/ → workers/industry/manufacturing/
3.3 workers/bpo/clinic/ → workers/industry/clinic/
3.4 workers/bpo/nursing/ → workers/industry/nursing/
3.5 workers/bpo/realestate/ → workers/industry/realestate/
3.6 workers/bpo/logistics/ → workers/industry/logistics/
3.7 全ファイルのimportをsedで一括置換
3.8 PIPELINE_REGISTRY のモジュールパスを更新
3.9 テスト実行（全業界テスト）
```

sedコマンド例:

```
INDUSTRIES="construction manufacturing clinic nursing realestate logistics"
for ind in $INDUSTRIES; do
  find shachotwo-app/ -name "*.py" -exec sed -i '' \
    "s/from workers\.bpo\.${ind}/from workers.industry.${ind}/g" {} +
  find shachotwo-app/ -name "*.py" -exec sed -i '' \
    "s/import workers\.bpo\.${ind}/import workers.industry.${ind}/g" {} +
done
```

凍結業種も同時に移動:

```
FROZEN="dental hotel beauty auto_repair pharmacy ecommerce staffing architecture
professional restaurant"
for ind in $FROZEN; do
  find shachotwo-app/ -name "*.py" -exec sed -i '' \
    "s/from workers\.bpo\.${ind}/from workers.industry.${ind}/g" {} +
done
```

検証コマンド:

```
cd shachotwo-app
python -c "from workers.industry.construction.pipelines import estimation_pipeline"
python -c "from workers.industry.manufacturing.pipelines import quoting_pipeline"
pytest tests/workers/bpo/construction/ tests/workers/bpo/manufacturing/ -v --tb=short
```

コミット: `refactor: move all industry plugins to workers/industry/ (Step 3/8)`

Step 4: 営業パイプラインの移動

```
4.1 workers/bpo/sales/ → workers/base/sales/
4.2 importをsedで一括置換
4.3 PIPELINE_REGISTRYを更新
4.4 routers/sales.py, marketing.py, crm.py 等のimportを更新
4.5 main.pyのscheduler importを更新
4.6 テスト実行（営業テスト）
```

sedコマンド例:

```
find shachotwo-app/ -name "*.py" -exec sed -i '' \
  's/from workers\.bpo\.sales/from workers.base.sales/g' {} +
```

検証コマンド:

```
cd shachotwo-app
pytest tests/workers/bpo/sales/ -v --tb=short
```

コミット: `refactor: move sales pipelines to workers/base/sales/ (Step 4/8)`

Step 5: バックオフィスパイプラインの再編

```
5.1 workers/bpo/common/pipelines/ を機能別に分割:
    → workers/base/backoffice/accounting/ (8本)
    → workers/base/backoffice/labor/ (5本)
    → workers/base/backoffice/hr/ (3本)
    → workers/base/backoffice/admin/ (3本)
    → workers/base/backoffice/procurement/ (2本)
    → workers/base/backoffice/legal/ (2本)
    → workers/base/backoffice/it/ (1本)
5.2 importをsedで一括置換
5.3 PIPELINE_REGISTRYを更新
5.4 routers/bpo/common.py, routers/bpo/backoffice.pyのimportを更新
5.5 テスト実行（バックオフィステスト）
```

注意: このステップは最も分割数が多い。1カテゴリずつ移動し、その都度テストを実行すること。

検証コマンド:

```
cd shachotwo-app
python -c "from workers.base.backoffice.accounting import invoice_pipeline"
pytest tests/workers/bpo/test_attendance_pipeline.py -v --tb=short
pytest tests/workers/bpo/test_contract_pipeline.py -v --tb=short
```

コミット: `refactor: reorganize backoffice pipelines into functional dirs (Step 5/8)`

Step 6: PIPELINE_REGISTRYの2層化

```
6.1 PIPELINE_REGISTRY → BASE_PIPELINES + INDUSTRY_PIPELINES に分割
6.2 route_and_execute を2層チェックに変更
6.3 LEGACY_KEY_MAP を追加（旧キー互換）
6.4 テスト実行（全テスト）
```

LEGACY_KEY_MAP の例:

```
LEGACY_KEY_MAP = {
    "bpo.construction.estimation": "industry.construction.estimation",
    "bpo.manufacturing.quoting": "industry.manufacturing.quoting",
    "bpo.common.attendance": "base.backoffice.labor.attendance",
    # ... 全旧キーを網羅
}
```

検証コマンド:

```
cd shachotwo-app
pytest tests/ -v --tb=short 2>&1 | tail -20
```

コミット: `refactor: split PIPELINE_REGISTRY into BASE + INDUSTRY (Step 6/8)`

Step 7: ルーターの整理

```
7.1 routers/bpo/construction.py → routers/industry/construction.py（オプション）
7.2 routers/bpo/common.py + routers/bpo/backoffice.py → routers/backoffice.py に統合
7.3 main.py の include_router を更新
7.4 テスト実行（全テスト）
```

注意: APIパスが変わる場合はフロントエンドも更新が必要。
APIパスを変えない（prefix据え置き）なら影響はimportのみ。

検証コマンド:

```
cd shachotwo-app
python -c "from main import app; print([r.path for r in app.routes][:10])"
pytest tests/routers/ -v --tb=short
```

コミット: `refactor: reorganize routers to match new layer structure (Step 7/8)`

Step 8: 旧ディレクトリの削除

```
8.1 workers/bpo/ が空であることを確認
8.2 workers/bpo/ を削除
8.3 全テスト実行（最終確認）
8.4 PRレビュー + マージ
```

確認コマンド:

```
# bpo/ 配下に残っているファイルがないことを確認
find shachotwo-app/workers/bpo/ -name "*.py" -not -name "__init__.py" | head -20

# 全テスト実行
cd shachotwo-app
pytest tests/ -v --tb=short 2>&1 | grep -E "PASSED|FAILED|ERROR" | tail -30

# importで旧パスを参照しているファイルがないことを確認
grep -r "from workers\.bpo\." shachotwo-app/ --include="*.py" | grep -v __pycache__
grep -r "import workers\.bpo\." shachotwo-app/ --include="*.py" | grep -v __pycache__
```

コミット: `refactor: remove legacy workers/bpo/ directory (Step 8/8)`

各ステップの所要時間見積り

Step	内容	見積り
1	Manager+Engine移動	2時間
2	micro業界中立化	3時間
3	業界プラグイン移動	2時間
4	営業パイプライン移動	2時間
5	バックオフィス再編	3時間
6	REGISTRY 2層化	2時間
7	ルーター整理	1時間
8	旧ディレクトリ削除	0.5時間
合計		**15.5時間（2日）**

ロールバック手順

各ステップ完了時にgit commitする。問題が発生した場合:

```
git stash           # 現在の変更を退避
git log --oneline   # 最後の安定commitを確認
git reset --hard <safe_commit>  # 安定状態に戻す
```

Step別ロールバック

Step	ロールバック難易度	方法
1	低	git revert 1コミット
2	中	compliance.pyの復元 + rules/削除
3	低	git revert 1コミット（import置換の逆）
4	低	git revert 1コミット
5	中	ファイル分割の逆統合が必要
6	低	REGISTRY定義の復元のみ
7	低	git revert 1コミット
8	N/A	削除済みディレクトリは git で復元可能

全体ロールバック（最悪ケース）

```
git checkout main
git branch -D refactor/3layer-architecture
```

mainブランチには一切影響しないため、最悪の場合はブランチごと破棄して最初からやり直せる。

完了条件

- [ ] 全テスト: 失敗数がベースライン（22件=凍結業種分）以下
- [ ] 新規テスト失敗: 0件
- [ ] `workers/bpo/` ディレクトリが存在しない
- [ ] PIPELINE_REGISTRY が BASE_PIPELINES + INDUSTRY_PIPELINES に分離
- [ ] micro/compliance.py に業界固有コードが存在しない
- [ ] `grep -r "from workers.bpo." shachotwo-app/` の結果が0件
- [ ] フロントエンドのAPI呼び出しが全て正常動作
- [ ] PRレビュー完了・mainにマージ済み